Отчет

Подготовила: Сиднева Ксения Владимировна

Группа: 11-412

Репозиторий: https://github.com/ksenyaaa/ml-projects.git

Структура

part1_ml_tasks/Серия_1.ipynb — Jupyter Notebook с 4 ML-задачами:

- Sentiment Analysis (RuBERT)
- ASR (Whisper-tiny)
- Image Classification (EfficientNetB0)
- Object Detection (YOLOv8n)

part2_fastapi/main.py — FastAPI приложение с эндпоинтами / и /predict

part2_fastapi/test_main.py — тесты для API

part2_fastapi/requirements.txt — зависимости проекта

part2_fastapi/.github/workflows/test.yml — CI/CD пайплайн для GitHub Actions

part3_recsys/Серия_3.ipynb — Jupyter Notebook с рекомендательной системой (MovieLens 100K)

В проекте реализованы четыре ML-темы: Sentiment Analysis, ASR, Image Classification и Object Detection. REST API для NLP-модели с CI/CD пайплайном и рекомендательная система на датасете MovieLens 100K.

Стек

Стек технологий

- Python, FastAPI, Uvicorn, Pydantic v2: HTTP API, схемы и валидация входа/выхода.
- Transformers и PyTorch: Sentiment Analysis на `blanchefort/rubert-base-cased-sentiment`; ASR на `openai/whisper-tiny`.
- TensorFlow/Keras Applications, Ultralytics: Image Classification на ` EfficientNetB0`; Object Detection на `YOLOv8n`.
- pandas, NumPy, scikit-learn: подготовка каталога и алгоритмы RecSys (TF-IDF, cosine similarity).
- pytest, FastAPI TestClient: unit/API-проверки; GitHub Actions: запуск `pytest` на push/PR.

Преимущества выбранных библиотек

Hugging Face Transformers — единый интерфейс pipeline() для разных задач, хаб из 500 000+ моделей, включая русскоязычные (RuBERT, Whisper). Интеграция с PyTorch и TensorFlow.

TensorFlow / Keras Applications — готовые предобученные модели (EfficientNetB0) с весами ImageNet, высокоуровневый API, встроенные функции preprocess_input и decode_predictions для быстрой подготовки данных и интерпретации результатов.

Ultralytics YOLOv8 — самая быстрая однопроходная детекция, простой API (YOLO('yolov8n.pt')), экспорт в ONNX/TensorRT.

FastAPI + Pydantic — автоматическая валидация данных, встроенный Swagger UI, асинхронность, интеграция с pytest.

scikit-learn — TF-IDF, косинусное сходство, матричные разложения из коробки, хорошо документированные метрики.

Sentiment Analysis (NLP)

Цель: классифицировать русскоязычный текст по эмоциональной окраске (POSITIVE / NEGATIVE / NEUTRAL). Используется pipeline sentiment-analysis с моделью RuBERT, дообученной на русскоязычных отзывах.

Принцип работы модели: RuBERT — русскоязычная версия BERT с механизмом Self-Attention, позволяющим учитывать контекст всего предложения. На вход подаётся токенизированный текст. Выходное представление токена проходит через классификатор, выдающий вероятности трёх классов.

```
classifier = pipeline("sentiment-analysis", model="blanchefort/rubert-base-cased-sentiment")

results = classifier(texts)
```

Результаты на тестовых примерах:

```
Текст: 'Этот курс просто потрясающий, я узнал очень много нового!'
Результат: POSITIVE (Уверенность: 90.60%)

Текст: 'Ужасный сервис, ничего не работает, деньги на ветер.'
Результат: NEGATIVE (Уверенность: 75.15%)

Текст: 'Я сегодня ходил в магазин, купил мороженое.'
Результат: NEUTRAL (Уверенность: 81.09%)

Текст: 'Я люблю учится, но мне безумно скучно сидеть на парах.'
Результат: NEGATIVE (Уверенность: 75.16%)
```

Анализ: модель корректно определяет явные позитивные и негативные маркеры. Особый интерес представляет последний пример с союзом «но» — модель правильно интерпретирует, что итоговая эмоциональная окраска определяется второй частью предложения («скучно»), несмотря на наличие позитивного начала («люблю учиться»). Уверенность в таких пограничных случаях обычно снижается (75.16% против 90.60% для чистого позитива), что отражает внутреннюю неопределённость модели при смешанной тональности.

Метрики: в поставке есть тестовые сэмплы. Для оценки следует публиковать Accuracy, F1-score по классам, матрицу ошибок (confusion matrix) и latency на CPU.

ASR (Audio)

Цель: транскрибировать аудиофайл в текст. Используется архитектура Encoder-Decoder (Whisper-tiny), обученная на 680k часах многоязычных данных.

Принцип работы: аудиосигнал преобразуется в mel-спектрограмму. Encoder извлекает акустические признаки, Decoder авторегрессивно генерирует текстовые токены.

```
whisper_pipeline = pipeline(
    "automatic-speech-recognition",
    model="openai/whisper-tiny"
)

result1 = whisper_pipeline(audio1, language="english", task="transcribe")
```

Результаты:

```
распознанный текст:  He hoped there would be stew for dinner, turnips and carrots and bruised potatoes and fat mutton pieces to be ladled out in thick,

распознанный текст:  Мне очень нравится учиться и я очень люблю изучать что-то новое.
```

Анализ: модель whisper-tiny (39 млн параметров) успешно справилась как с чистым английским аудио из официального датасета, так и с реальным русскоязычным голосовым сообщением из мессенджера. Это демонстрирует универсальность архитектуры и её способность работать в условиях естественной речи с паузами и шумами.

Метрики: WER (Word Error Rate) ~10–15% для whisper-tiny на чистых записях. Для русского языка дополнительно рекомендуется CER (Character Error Rate).

Image Classification (CV)

Цель: определить объект на статичном изображении. Используется EfficientNetB0 с инвертированными остаточными блоками и depthwise separable свёртками, предобученная на ImageNet (1000 классов).

Принцип работы: EfficientNetB0 использует составное масштабирование (compound scaling), которое балансирует глубину, ширину и разрешение сети. Входное изображение 224×224 проходит через серию MBConv блоков, извлекающих признаки от простых (края, текстуры) к сложным (объекты). Выходной слой — softmax на 1000 классов ImageNet.

```
img_array = img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

preds = model.predict(img_array)
```

Данные/метрики: Top-1 Accuracy на ImageNet ~71.8%. На тестовом изображении модель выдала корректный результат: первое место — `lynx` (50.27%), в Top-3 вошли `tabby` и `Egyptian_cat`, что подтверждает извлечение признака «семейство кошачьих».



```
топ-3 предсказания модели:
- tiger_cat (ID: n02123159): 45.62%
- tabby (ID: n02123045): 21.18%
- Egyptian_cat (ID: n02124075): 9.04%
```

Анализ: модель корректно классифицировала изображение. Все три предсказания относятся к семейству кошачьих, что демонстрирует высокую семантическую точность модели. Суммарная уверенность по всем кошачьим классам составляет 75.84%, что указывает на высокую уверенность модели в правильном выборе категории объекта.

Метрики: Top-1 Accuracy на ImageNet ~77.1%, Top-5 Accuracy ~93.5%.

Object Detection (CV)

Цель: найти и локализовать объекты в кадре с помощью bounding boxes. Используется архитектура YOLOv8n (You Only Look Once) — однопроходная детекция.

Принцип работы: изображение делится на сетку, для каждой ячейки предсказывается K bounding boxes с координатами, уверенностью (objectness score) и вероятностями классов. Используется anchor-free подход и NMS для удаления дублирующихся рамок.

```
results = model.predict(source=img_path, show=False, save=True)

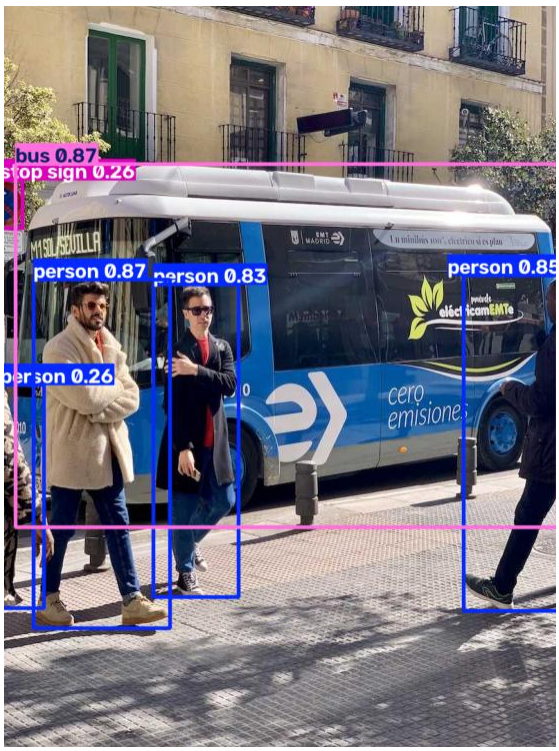
if len(results[0].boxes) > 0:
    boxes = results[0].boxes
    print(f"найдено объектов: {len(boxes)}")
    print("\nнайденные объекты:")
    for i, (cls, conf) in enumerate(zip(boxes.cls, boxes.conf)):
        print(f"- {model.names[int(cls)]}: {conf*100:.1f}%")
else:
    print("объекты не найдены")
```

Результаты:

```
найденные объекты:
- bus: 87.3%
- person: 86.6%
- person: 85.3%
- person: 82.5%
- person: 26.1%
- stop sign: 25.5%
```

Анализ: модель корректно обнаружила основные объекты сцены. Объекты с низкой уверенностью (26.1% и 25.5%) — это частично видимые или дальние объекты. В production-системах такие детекции обычно фильтруются порогом confidence ≥ 0.5.

Метрики: mAP@50 ~37.3% (COCO benchmark). Скорость инференса ~40–60 FPS на CPU, что позволяет использовать модель для real-time видео.



API

Модель Sentiment Analysis обёрнута в REST API. Эндпоинт `POST /predict` принимает JSON, валидирует его через Pydantic и возвращает `{"label", "confidence"}`. Написаны модульные тесты на PyTest с использованием `TestClient`. Тесты проверяют HTTP-статус 200 и логическую корректность ответа (`assert data["label"] == "NEGATIVE"`). Настроен CI/CD pipeline через GitHub Actions: при каждом push автоматически разворачивается Ubuntu-среда и запускается `pytest -v`.

```python
class TextRequest(BaseModel):
    text: str

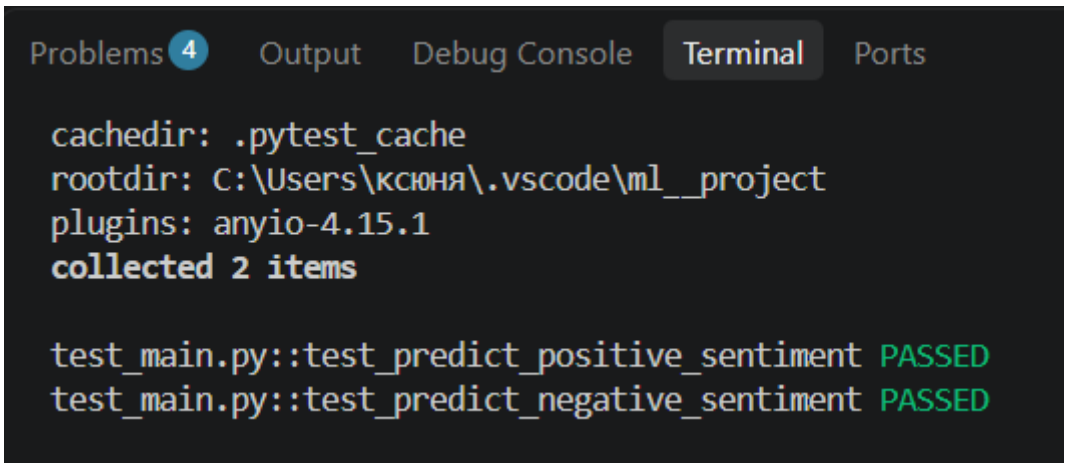
sentiment_pipeline = pipeline("sentiment-analysis", model="blanchefort/rubert-base-cased-sentiment")

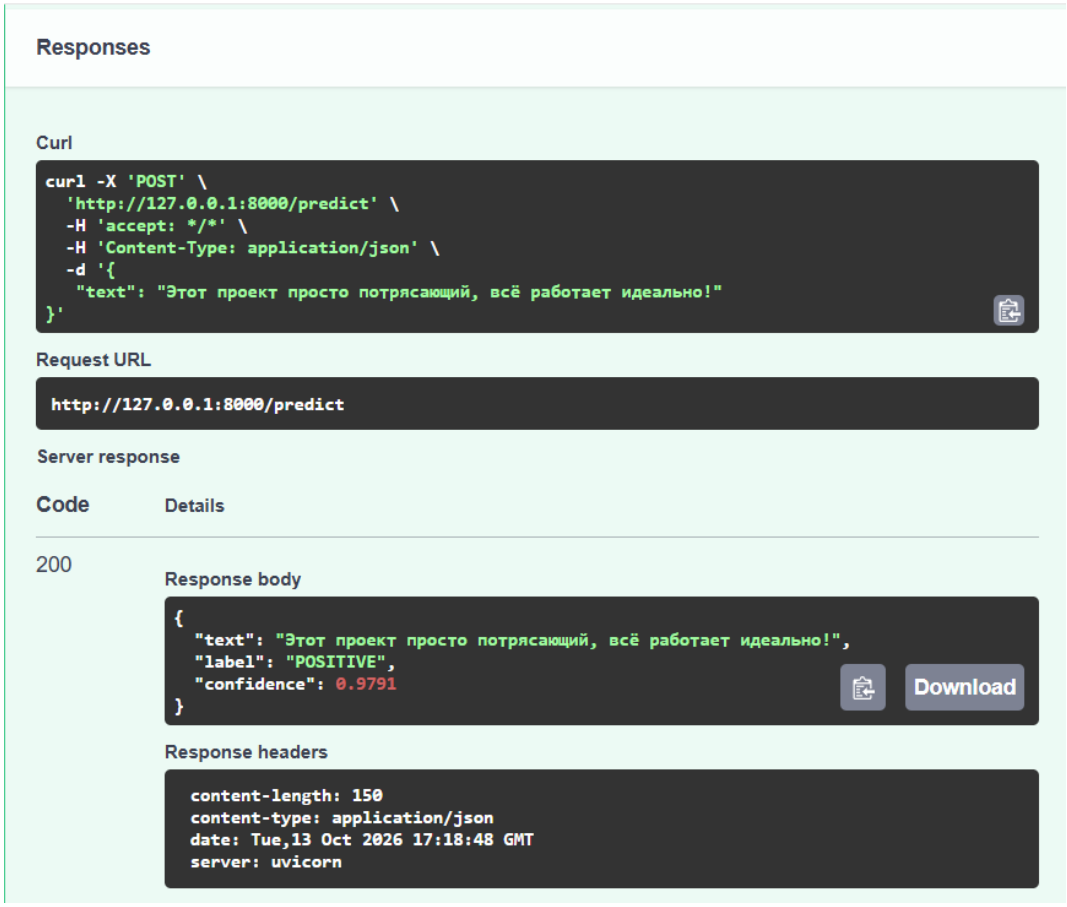
@app.post("/predict")
def predict_sentiment(request: TextRequest):
    input_text = request.text

    result = sentiment_pipeline(input_text)[0]

    return {
        "text": input_text,
        "label": result["label"],
        "confidence": round(result["score"], 4)
    }
```

Проверка





RecSys

Локальный датасет MovieLens 100K содержит 100 000 оценок от 943 пользователей для 1682 фильмов. Формат оценок: целые числа от 1 до 5. Каждый фильм описан 19 жанровыми тегами.

Реализованные подходы:

- Popularity-based (heuristic): сортировка по среднему рейтингу с фильтром min_count >= 50. Используется как fallback для Cold Start.

Popularity-based (мин. 50 оценок):

	title	avg_rating	count
0	Close Shave, A (1995)	4.491071	112
1	Schindler's List (1993)	4.466443	298
2	Wrong Trousers, The (1993)	4.466102	118
3	Casablanca (1942)	4.456790	243
4	Wallace & Gromit: The Best of Aardman Animatio...	4.447761	67

- Genre-based (heuristic): подбор фильмов с максимальным совпадением жанровых тегов.

Genre-based для 'Toy Story (1995)':

	title	match_count
819	Space Jam (1996)	3
94	Aladdin (1992)	3
421	Aladdin and the King of Thieves (1996)	3
992	Hercules (1997)	3
1218	Goofy Movie, A (1995)	3

- Content-based: извлекает жанровые теги, строит TF-IDF матрицу, рассчитывает cosine similarity. Решает проблему Cold Start для новых фильмов (item cold start).

Content-based рекомендации для 'Toy Story (1995)':

	title
421	Aladdin and the King of Thieves (1996)
101	Aristocats, The (1970)
403	Pinocchio (1940)
624	Sword in the Stone, The (1963)
945	Fox and the Hound, The (1981)

- Collaborative filtering (Item-based): строит user-item матрицу рейтингов, рассчитывает cosine similarity между фильмами на основе паттернов оценок пользователей.

Collaborative filtering для 'Toy Story (1995)':

	title
49	Star Wars (1977)
116	Rock, The (1996)
120	Independence Day (ID4) (1996)
180	Return of the Jedi (1983)
404	Mission: Impossible (1996)



Проблемы и ограничения: рассмотрены Cold Start (пользователь/фильм), Sparsity (разреженность матрицы), Filter Bubble (решается через Explore/Exploit стратегию) и Production-ограничения (latency решается через pre-compute и кэширование).

Метрики оценки

ML-метрики: Precision@K, Recall@K, NDCG (учитывает порядок выдачи), RMSE.

Бизнес-метрики: CTR (кликабельность), Watch Time, Conversion Rate, Retention Rate.

Выводы

Проект реализован в одиночку. Решено было реализовать задачи в 4 разных сферах (NLP, Audio, CV, RecSys), чтобы рассмотреть различные архитектуры (Transformer Encoder, Encoder-Decoder, CNN, One-stage detector). Сталкивались с трудностями при установке зависимостей (несовместимость Python 3.14 и старых версий torch) и при скачивании аудио-датасетов (ошибки 404 на прямых ссылках), вот с ними прямо пришлось повозится, так как русское аудио было сложно найти, но в итоге это было решено через

альтернативные способы загрузки (библиотека `datasets` в режиме streaming, загрузка .ogg из Telegram).